

Problem A. Zeros

We are interested in computing the number of zeros at the end of the number $\text{lcm}(a, a+1, \dots, b)$, that is, of the least common multiple of numbers $a, a+1, \dots, b$.

Subtask 1

When $b \leq 16$, we can explicitly compute the lcm that Santa is interested in. The maximum value that it can attain is $\text{lcm}(1, 2, \dots, 16) = 16 \cdot 9 \cdot 5 \cdot 7 \cdot 11 \cdot 13 = 720\,720$.

One way is to use the facts that $\text{lcm}(x, y, z) = \text{lcm}(\text{lcm}(x, y), z)$ and that $\text{lcm}(x, y) = \frac{x \cdot y}{\text{gcd}(x, y)}$, where $\text{gcd}(x, y)$ is the greatest common divisor of x and y and can be computed using a built-in library function (e.g. `__gcd` in C++) or using the Euclidean algorithm.

Another possibility is to compute the lcm directly from the definition – loop over every positive integer and check if it is a multiple of all of $a, a+1, \dots, b$. (Return the first number that passes this test.)

Once we have computed the lcm, we can find its number of trailing zeros by dividing it by 10 as many times as possible.

Subtask 2

Here we can still compute the lcm explicitly. However, we must use the first method above, as now the maximum value is $\text{lcm}(1, 2, \dots, 40) = 5\,342\,931\,457\,063\,200 \approx 5 \cdot 10^{15}$ and we cannot afford to make this many iterations. Moreover, we must take care to use a 64-bit integer variable type (such as `long long` in C++).

Subtask 3

One solution is to still compute the lcm explicitly – but now the result can have up to 90 digits, so we need to either implement our own big-number type, or use Python, which has that built in. However, there is a much more elegant solution, which consists only of several `if` statements.

We can see that the answer is 0 for $b = 1$, it can only increase as b grows, and it only increases at powers of 5, by 1 at a time. Thus we can answer 0 for $b \in \{1, 2, 3, 4\}$, 1 for $b \in \{5, \dots, 24\}$, 2 for $b \in \{25, \dots, 124\}$, and 3 for $b \in \{125, \dots, 200\}$. But why is that? To see this, let us make some simple observations (which will be useful also for the following subtasks).

For two positive integers q and x , let us denote by $v_q(x)$ the multiplicity of q as a divisor of x , that is, the maximum $k \in \mathbb{Z}_{\geq 0}$ such that q^k divides x . Then we have:

- $v_{10}(x)$ is the number of zeros at the end of x .
- $v_{10}(x) = \min(v_2(x), v_5(x))$. This is because $10^k \mid x$ if and only if $2^k \mid x$ and $5^k \mid x$.
- For a prime number p , $v_p(\text{lcm}(x, y)) = \max(v_p(x), v_p(y))$. This is because if we decompose both x and y into prime factors: $x = p_1^{k_1} \cdot \dots \cdot p_s^{k_s}$ and $y = p_1^{\ell_1} \cdot \dots \cdot p_s^{\ell_s}$, where $p_1, \dots, p_s$ are different primes and $k_1, \dots, k_s, \ell_1, \dots, \ell_s \in \mathbb{Z}_{\geq 0}$, then $\text{lcm}(x, y) = p_1^{\max(k_1, \ell_1)} \cdot \dots \cdot p_s^{\max(k_s, \ell_s)}$.
- As a consequence,

$$v_{10}(\text{lcm}(a, a+1, \dots, b)) = \min[\max(v_2(a), v_2(a+1), \dots, v_2(b)), \max(v_5(a), v_5(a+1), \dots, v_5(b))].$$

For this subtask, the solution follows by noting that when $a = 1$, then $\max(v_2(1), v_2(2), \dots, v_2(b))$ is the largest k such that $2^k \leq b$ (in other words, it is $\lfloor \log_2 b \rfloor$). The term corresponding to 5 is the largest k such that $5^k \leq b$, and of course this second k is no larger, as $2^k \leq 5^k$ (equivalently, $\log_5 b \leq \log_2 b$). This value of k is 0 for $b \in \{1, \dots, 5-1\}$, 1 for $b \in \{5, \dots, 5^2-1\}$, 2 for $b \in \{5^2, \dots, 5^3-1\}$, and 3 for $b \in \{5^3, \dots, 200\}$ (since $200 < 5^4$).

Subtask 4

Now the input numbers are becoming large (remember to read them as 64-bit integers). However, the condition $b - a \leq 10^6$ means that we can use the $\min[\max(\dots), \max(\dots)]$ formula above explicitly, by

looping over all $i = a, \dots, b$. For each such i we compute $v_2(i)$ directly, by dividing i by 2 as many times as possible; this will take at most $\log_2(10^{18})$ steps; and similarly for 5.

Subtask 5

We simply generalize the solution of subtask 3 above: instead of several **if** statements, we compute $k = \lfloor \log_5 b \rfloor$ by starting from $k = 0$ and increasing it as long as $5^k \leq b$.

Subtask 6

To solve the general task, we will compute the expression $\max(v_2(a), v_2(a+1), \dots, v_2(b))$ (and similarly for 5) faster than in $O(b-a)$ time. That is, we want to find the largest k such that 2^k divides some number $i \in \{a, \dots, b\}$. Clearly, this reduces to checking whether a given k is good, as there are only at most $\log_2(10^{18})$ different k -values to check. To find out whether the interval $\{a, \dots, b\}$ contains some multiple of 2^k , we can, for example, round b down to the nearest multiple of 2^k , and check if a is still no larger than the rounded b . More formally, we compute the largest multiple of 2^k that is at most b (this is $\lfloor \frac{b}{2^k} \rfloor \cdot 2^k$) and check whether it is no smaller than a . If yes, then it is a multiple of 2^k that lies in the interval $\{a, \dots, b\}$; otherwise, there is no such multiple.

Problem B. Luna Likes Love

The solution of Subtask 1 is straightforward – since there is no person between friends forming a couple, we can send all n couples to a date without having to do any swaps. This requires n actions.

Similarly, in Subtask 2, we process friends from left to right. There are two cases. One case is that the leftmost person has their partner next to them. In this case we can just send them to a date. Otherwise, the partner of the leftmost person must be third in the row, and the second and fourth person must form another couple. (This is because the second person must also have at most one person between them and their partner.) In this case we clearly must do one swap and then we can send both pairs on a date.

We can now note that the observation we just made can be generalized as follows: whenever you have two couples X and Y who are standing in the row like this: “ $X...Y...X...Y$ ”, you will, sooner or later, have to swap the inner Y and X . Before doing so, you won’t be able to send either X s or Y s away. Thus, the total number of actions is at least equal to n + the number of pairs which overlap in this way. Below we will show that this number of actions is always sufficient.

Subtasks 3 and 4 offer a special case in which it is easier to discover this property of the problem, and it is also easier to count the swaps needed.

We will now show that there is always a solution in which the number of swaps equals the number of overlapping pairs. Suppose we have a row of friends. Let’s find a couple that’s currently closest to each other: $a_p = a_q$ and $q - p$ is as small as possible. From the minimality of $q - p$ it follows that the people who are standing between positions p and q must belong to distinct couples. Each of those couples therefore has an overlap with our chosen couple. We can now use swaps to bring the person in position q next to the person with position p and then we send them to a date. The number of swaps we made is exactly equal to the number of overlaps that involved our couple. Once we are done, the other people are still in the same relative order, so the other overlapping pairs remained unchanged. Clearly, repeating this process will give us a valid solution, and the total number of swaps made will be exactly equal to the total number of overlapping couples.

This strategy can actually be implemented and it is one possible way to completely solve the task. However, an easier implementation is also possible. Remember that we don’t have to actually produce a sequence of actions, we just have to compute the minimum total number of actions. In other words, we just need to count the number of pairs of couples which have a partial overlap.

This is really easy to do in $O(n^2)$. For each of the n couples, go over the people between them and count the number of occurrences of each value. The values that occur once are other couples that have an overlap with this one. (This counts each overlap twice.)

In order to solve the last subtask, we will count the overlapping pairs in $O(n \log n)$ time.

For each couple X we want to count overlaps of the form “ $X...Y...X...Y$ ” – i.e., those in which the first person of the other couple is between them and the other one only appears later. This way we will count each overlap once.

We will process the input sequence from the left to the right. Whenever we encounter the first person of a couple, they will start being active, and whenever we encounter the second person of a couple, the first person of that couple will stop being active. Additionally, whenever a couple ends, we want to count the overlaps. These correspond precisely to the active people who stand between our couple.

We can use a Fenwick tree or an interval tree to keep track of the active people. Both data structures offer us the option to switch a person from inactive to active or back in $O(\log n)$ time, and also the option to count active people within a given range in $O(\log n)$ time.

Problem C. Twin Cookies

Subtask 1

You can output what is given in the sample. Alternatively, ask 1, 2 and 3.

Subtask 2

First ask 1 and 2. Let the returned value be x .

Then ask 10 and 20. Let the returned value be y .

Then, ask $y - x$ and $y + x$. If $y - x$ is returned, give x and $y - x$ to the first child, and y to the second. If $y + x$ is returned, give x and y to the first child, and $y + x$ to the second.

Subtask 3

For $i \in \{0, \dots, 49\}$, first ask $2ni + (1 \dots n)$, then $2ni + (n + 1 \dots 2n)$. Let x_i be the first value returned, and y_i the second. Define $d_i = y_i - x_i$.

We have $1 \leq d_i \leq 2n - 1$. Thus, for $n \leq 25$, by the pigeonhole principle, there must exist some pair $i, j \in \{0, \dots, 49\}$ of distinct indices such that $d_i = d_j$.

Now, give x_i and y_j to the first child, and x_j , y_i to the second.

Subtasks 4-6

The expected solutions to the larger subtasks all employ the same idea, also based on the pigeonhole principle, at differing levels of optimisation.

If we perform k queries, there are 2^k possible subset sums we can create with the k returned values. On the other hand, as long as all values we ask are at most H , the sum of any subset is at most kH . We can achieve $H = kn$ by including the first n unused numbers in every query.

Thus, as long as $2^k > k^2n$, we can guarantee that some two subsets have the same sum. Even for $n = 5000$, this is achieved for $k \geq 22$.

Given any two different subsets S_1 and S_2 with the same sum, we can now remove $S_1 \cap S_2$ from both. Removing only elements in both sets keeps the sets distinct and their sums equal, so giving the values in $S_1 \setminus (S_1 \cap S_2)$ to the first child, and values in $S_2 \setminus (S_1 \cap S_2)$ to the second is a solution.

It only remains to compute two such subsets S_1 and S_2 .

Subtask 4

For subtask 4, we can maintain for every sum $s \leq 101^2 \cdot n$ up to one subset S_s that achieves that sum. After one of our queries gives us the value x to work with, we update these subsets. For every sum s , we see if we can achieve sum $s - x$. If we can achieve that sum, the subset $S_{s-x} \cup \{x\}$ achieves sum s . If the sum s was already achievable, we have found a pair of two different subsets of equal sum. Otherwise, we let $S_s \leftarrow S_{s-x} \cup \{x\}$. Note that we have to make the updates in order of decreasing s , to not create sets that contain multiple copies of x .

Maintaining the subsets as vectors, we do $\mathcal{O}(k \cdot k^2n \cdot k)$ work. For $k \leq 22$ and $n \leq 200$, this is fast enough.

Subtask 5

Instead of storing the set S_s for every sum s , we store the value x that let us make the set $S_s = S_{s-x} \cup \{x\}$. To actually construct the set at sum s , we can then iteratively add x to the set, and subtract x from s .

Since we do not have to work with vectors, we trim one factor of k , giving a $\mathcal{O}(k \cdot k^2n)$ algorithm.

Subtask 6

A fast implementation of the $\mathcal{O}(k^3n)$ algorithm for subtask 5 can pass even subtask 6. However, faster solutions exist as well:

We can still trim one factor of k : notice that as long as no sum is achieved by two different subsets, there are exactly 2^t achievable sums after adding t numbers. Thus, if we maintain a sorted vector C_i of achievable sums at step i , we can compute $C_{i+1} = C_i \cup (C_i + x_i)$ in linear time to $|C_i|$ by merging two

sorted vectors, and thus we can compute all C_i in $\mathcal{O}(k^2n)$ work total.

Given an achievable sum s at step i , we can easily compute a set corresponding to it: if $s - x_i \in C_{i-1}$, find the set corresponding to sum $s - x_i$ and step $i - 1$ and add x_i to it, otherwise find the set corresponding to sum s and step $i - 1$.

Given the first sum s and step i that is achievable in two ways, we can then construct the two distinct sets of equal sum by constructing $s - x_i$ and s at step $i - 1$, and adding x_i to the first.

Alternatively, a $\mathcal{O}(k^3n/\log n)$ solution could be achieved using bitsets. We maintain bitsets B_i with k^2n bits, with a 1 at position s if the sum s is achievable after adding the first i values x_i we receive (so B_i is a bitset representation of C_i). We have $B_{i+1} = B_i \parallel B_i \ll x_i$, where $\parallel$ is the bitwise OR operation, and $\ll$ the right shift operation. Using AND instead of OR lets us check if any sum can be achieved in two ways. Once we find such a sum and step, we construct the solution as before.

By changing from the first approach to the second after $k - \log \log n$ steps, we could have complexity $\mathcal{O}(k^2n \log \log n / \log n)$ with the same memory usage, but this is wildly unnecessary for this problem.

Problem D. Lanterns

Subtask 1

In the first subtask, the limits are very low, and almost any brute force solution passes. One solution is as follows. Consider a fixed permutation of lanterns. Note that we can always buy the first one by definition.

Assume that we have already bought the first j lanterns. We can easily find the range of peaks that are reachable from the starting position using these lanterns. If all peaks are reachable, we are done and the answer for this starting point and this permutation of lanterns equals the total cost of the first j lanterns. Otherwise, check that the $j + 1$ -th lantern is reachable. If it isn't, or we have already bought all the lanterns, this particular permutation doesn't yield any solution. Otherwise, repeat the same process.

The complexity of this approach is $\mathcal{O}(k!(k + n))$.

Subtask 2

Let L be a set of purchased lanterns. The set of reachable altitudes is the union of all $[a_j, b_j]$ for all lanterns $j \in L$.

For the subtasks, we need the following simple, yet important observation:

There is always an optimal solution in which at any point in time the set of reachable altitudes forms a contiguous interval.

Indeed, this is clearly satisfied when the first lantern is purchased, and all purchases that would violate the constraint can be deferred to a later time. This is because we can never leave the interval of reachable altitudes that we're currently in.

This suggests that dynamic programming approaches might work for this task.

Indeed, we can define our state as

$$D[l][h][cur] = d$$

where l is the lowest reachable altitude, h is the highest reachable altitude, cur is the current peak, and d is the minimum cost needed to reach all the peaks from this situation.

Initially, $D[1][n][x] = 0$ for all x from 1 to n .

To compute all the DP transitions, simply note that we can find the range $[c, d]$ of all the reachable peaks trivially in time $\mathcal{O}(n)$, and then for each lantern j that satisfies the following conditions:

- it can be purchased in one of peaks $[c, d]$
- its operational range is not disjoint from $[l, h]$
- its operational range is not a subrange of $[l, h]$ (i.e., buying it enlarges the intervals of reachable altitudes)

we set

$$D[l][h][cur] = \min(D[l][h][cur], D[\min(a_j, l)][\max(b_j, h)] + c_j)$$

For a lantern j (if $a_j \leq h_{p_j} \leq b_j$), the answer is

$$D[a_j][b_j][p_j] + c_j$$

There are $\mathcal{O}(k^2 \cdot n)$ states, and each of them can be processed in $\mathcal{O}(\max(k, n))$ time. The total time complexity is $\mathcal{O}(k^2 \cdot n \cdot \max(k, n))$ or $\mathcal{O}(n^4)$ with the assumption $n \approx k$.

The solution can be implemented as a recursive function with memoisation. Alternatively, one can see that the states can be processed in the order of increasing l and decreasing h .

Subtask 3-4

To improve the above solution, observe that we don't need the value of cur exactly – it is sufficient to just know the range of peaks we currently are. Thus, instead of remembering the lowest and highest reachable altitudes, we remember $[id_l, id_h]$: the ID of the lanterns that achieve the lowest and highest reachable altitude, respectively. Clearly, $l = a_{id_l}$ and $h = b_{id_h}$.

What makes this solution more efficient than the previous one is that we can set $cur = p_{id_l}$. This reduces the number of states to $\mathcal{O}(k^2)$.

The initial conditions, the solution reconstruction and the transitions can be performed the same way. As a result, the total time complexity is reduced to $\mathcal{O}(k^2 \max(k, n))$, or $\mathcal{O}(n^3)$ under the assumption that $n \approx k$.

Subtask 5

The full solution optimises the above approach even further.

Assume that the extremal lanterns are i, j and we buy the k -th lantern. Assume that the k -th lantern is reachable, and changes the lowest reachable altitude (but not the highest). The act of buying the k -th lantern is equivalent to setting, for each applicable i, j :

$$D[i][j] = \min(D[i][j], D[k][j] + c_k)$$

Let us fix k and j . We can express the above assignments for all valid choices of i as follows

$$D[i][j] = \min(D[i][j], \min_{k \in F_j^+(i)} D[k][j] + c_k),$$

where $F_j^+(i)$ is the set of all lanterns k that satisfy the following two conditions:

- $a_i \geq a_k$ (otherwise purchasing lantern k doesn't improve the range)
- Peak p_k is reachable when lanterns i and j are purchased.

The set $F_j(i)$ can have $\mathcal{O}(k)$ elements, so we cannot afford to compute it explicitly. Instead, we do the following.

We process the lanterns in decreasing order of a_i . This way, the first condition is satisfied automatically.

We store values of $D[i][j] + c_i$ in a segment tree that returns the minimum value in a range. In this segment tree, the lanterns are sorted by p_i . This way, the set of lanterns purchasable when i and j are the lanterns achieving the lowest and highest altitude, respectively, form an interval in this segment tree.

There are multiple ways of obtaining this interval. For a fixed pair of lanterns i, j , we look for the largest interval whose elevation is in the range $[a_i, b_j]$. We can find this interval using binary search on its endpoints, starting from p_i . In order to do that fast enough, we need to be able to answer range min and max queries on h . We can precompute these minimums and maximums in $\mathcal{O}(n^2)$ time.

To summarise:

- Precompute $lo_h(a, b) = \min_{i \in [a, b]} h(i)$ and $hi_h(a, b) = \max_{i \in [a, b]} h(i)$ for all $1 \leq a \leq b \leq n$ in $\mathcal{O}(n^2)$.
- Precompute $left(i, j)$, $right(i, j)$ – the range of reachable lanterns when lanterns i and j are purchased, using binary search and lo_h and hi_h .
- For a fixed j , store $D[i][j] + c_i$ in a segment tree, ordered by p_i .
- For each k , in order of increasing a_k , set $D[i][j] = \min(D[i][j], segtree.getmin(left(i, j), right(i, j)))$.

This handles the purchases of lanterns that decrease the lowest reachable altitude. We can create a similar set of segment trees for lanterns that increase the highest reachable altitude. Note that in this case, we have to process the lanterns in order of **decreasing** b_k .

Another complication is that there are also lanterns that decrease the lowest reachable altitude and increase the highest reachable altitude at the same time. We can use another segment tree to process these. The details of this last step are left as an exercise to the reader.

In total this solution uses $\mathcal{O}(\max(n, k)^2 \log \max(n, k))$ time.